

Name: Gaurika Mittal

Batch: 15

Sap ID: 590034892

Experiment-16

Question

Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer. Define **one separate user-defined function for each menu option**, with **void return type** and a **simple function call**.

Operations:

1. Generate the largest possible number using its digits.
2. Generate the smallest possible number using its digits.
3. Swap its first and last digits.
4. Check whether it is a palindrome number.

1. Algorithm

Main Program Algorithm

1. Start.
2. Display the menu:
 - Generate largest possible number
 - Generate smallest possible number
 - Swap first and last digits
 - Check palindrome
3. Input the user's choice.
4. Use `switch-case` to perform the selected operation.
5. For choice 1, call `largest()`.
6. For choice 2, call `smallest()`.
7. For choice 3, call `swapDigits()`.
8. For choice 4, call `palindrome()`.
9. For any other choice, display "Invalid choice".
10. Stop.

Algorithm for `largest()`

1. Start.

2. Input a positive integer.
3. Create an array `freq[10]` and initialize all elements to 0.
4. Extract each digit using `% 10`.
5. Increase the frequency of that digit.
6. Repeat until all digits are processed.
7. Starting from digit 9 down to 0, print each digit according to its frequency.
8. Stop.

Algorithm for `smallest()`

1. Start.
2. Input a positive integer.
3. Create an array `freq[10]` and initialize all elements to 0.
4. Extract each digit and store its frequency.
5. Find the smallest non-zero digit.
6. Print this digit first.
7. Print all zeros after it.
8. Print the remaining digits from 1 to 9 in increasing order.
9. Stop.

Note: We put the smallest non-zero digit first because a number should not start with 0.

Algorithm for `swapDigits()`

1. Start.
2. Input a positive integer.
3. Find the last digit using `% 10`.
4. Find the place value of the first digit.
5. Find the first digit.
6. Remove the first and last digits from their original positions.
7. Place the last digit at the first position.
8. Place the first digit at the last position.
9. Display the new number.
10. Stop.

Algorithm for `palindrome()`

1. Start.
2. Input a positive integer.
3. Store the original number.
4. Initialize `reverse = 0`.
5. Extract the last digit using `% 10`.
6. Add the digit to the reversed number.
7. Remove the last digit using `/ 10`.
8. Repeat until the number becomes 0.
9. Compare the reversed number with the original number.
10. If both are equal, display "Palindrome number".
11. Otherwise, display "Not a palindrome number".
12. Stop.

2. Test Case Table

Test Case	Choice	Inputs	Expected Outputs	Actual Outputs	Results
1	1	31542	Largest number = 54321	Largest number = 54321	Pass
2	1	92051	Largest number = 95210	Largest number = 95210	Pass
3	2	31542	Smallest number = 12345	Smallest number = 12345	Pass
4	2	50231	Smallest number = 10235	Smallest number = 10235	Pass
5	3	12345	Number after swapping = 52341	Number after swapping = 52341	Pass
6	3	9876	Number after swapping = 6879	Number after swapping = 6879	Pass
7	4	12321	Palindrome number	Palindrome number	Pass
8	4	12345	Not a palindrome number	Not a palindrome number	Pass

3. C Program

```
#include <stdio.h>
```

```
void largest();
void smallest();
void swapDigits();
void palindrome();
int main()
{
    int choice;
    printf("MENU\n");
    printf("1. Generate largest possible number\n");
    printf("2. Generate smallest possible number\n");
    printf("3. Swap first and last digits\n");
    printf("4. Check palindrome\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch(choice)
```

```

{
    case 1:
        largest();
        break;
    case 2:
        smallest();
        break;
    case 3:
        swapDigits();
        break;
    case 4:
        palindrome();
        break;
    default:
        printf("Invalid choice\n");
}
return 0;
}
void largest()
{
    long long n;
    int digit, freq[10] = {0};
    int i;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    while(n > 0)
    {
        digit = n % 10;
        freq[digit]++;
        n = n / 10;
    }
    printf("Largest number = ");
    for(i = 9; i >= 0; i--)
    {
        while(freq[i] > 0)
        {
            printf("%d", i);
            freq[i]--;
        }
    }

    printf("\n");
}

void smallest()
{
    long long n;
    int digit, freq[10] = {0};
    int i;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    while(n > 0)
    {
        digit = n % 10;
        freq[digit]++;
    }

```

```

        n = n / 10;
    }
    printf("Smallest number = ");
    for(i = 1; i <= 9; i++)
    {
        if(freq[i] > 0)
        {
            printf("%d", i);
            freq[i]--;
            break;
        }
    }
    while(freq[0] > 0)
    {
        printf("0");
        freq[0]--;
    }
    for(i = 1; i <= 9; i++)
    {
        while(freq[i] > 0)
        {
            printf("%d", i);
            freq[i]--;
        }
    }
    printf("\n");
}

void swapDigits()
{
    long long n, temp, first, last, place, result;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    last = n % 10;
    temp = n;
    place = 1;
    while(temp >= 10)
    {
        temp = temp / 10;
        place = place * 10;
    }
    first = temp;
    result = n - first * place - last;
    result = result + last * place + first;
    printf("Number after swapping = %lld\n", result);
}

void palindrome()
{
    long long n, original, reverse = 0;
    int digit;
    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    original = n;
    while(n > 0)
    {
        digit = n % 10;

```

```

        reverse = reverse * 10 + digit;
        n = n / 10;
    }
    if(original == reverse)
        printf("Palindrome number\n");
    else
        printf("Not a palindrome number\n");
}

```

4.Compilation and Execution

```

(kali㉿kali)-[~]
└─$ gcc hdoc17.c -o hdoc17

(kali㉿kali)-[~]
└─$ ./hdoc17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 1
Enter a positive integer: 31542
Largest number = 54321

(kali㉿kali)-[~]
└─$ gcc hdoc17.c -o hdoc17

(kali㉿kali)-[~]
└─$ ./hdoc17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 1
Enter a positive integer: 92051

```

```

Largest number = 95210

(kali㉿kali)-[~]
└─$ gcc hdoc17.c -o hdoc17

(kali㉿kali)-[~]
└─$ ./hdoc17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 2
Enter a positive integer: 31542
Smallest number = 12345

(kali㉿kali)-[~]
└─$ gcc hdoc17.c -o hdoc17

(kali㉿kali)-[~]
└─$ ./hdoc17
MENU
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 2
Enter a positive integer: 50231
Smallest number = 10235

```

```
(kali㉿kali)-[~]  
$ gcc hdoc17.c -o hdoc17
```

```
(kali㉿kali)-[~]  
$ ./hdoc17
```

MENU

1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome

Enter your choice: 4

Enter a positive integer: 12321

Palindrome number

```
(kali㉿kali)-[~]  
$ gcc hdoc17.c -o hdoc17
```

```
(kali㉿kali)-[~]  
$ ./hdoc17
```

MENU

1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome

Enter your choice: 4

Enter a positive integer: 12345

Not a palindrome number